

Every computer requires appropriate instruction set (programs) to perform the required task. The quality of the processing depends upon the given instructions. If the instructions are improper or incorrect, then it is obvious that the result will be superfluous.

Therefore, proper and correct instructions should be provided to the computer so that it can provide the desired output. Hence, a program should be developed in such a way that it ensures proper functionality of the computer. In addition, a program should be written in such a manner that it is easier to understand the underlying logic.

A good computer program should have following characteristics:

- **Portability:** Portability refers to the ability of an application to run on different platforms (operating systems) with or without minimal changes. Due to rapid development in the hardware and the software, nowadays platform change is a common phenomenon. Hence, if a program is developed for a particular platform, then the life span of the program is severely affected.
- **Readability:** The program should be written in such a way that it makes other programmers or users to follow the logic of the program without much effort. If a program is written structurally, it helps the programmers to understand their own program in a better way. Even if some computational efficiency needs to be sacrificed for better readability, it is advisable to use a more user-friendly approach, unless the processing of an application is of utmost importance.
- **Efficiency:** Every program requires certain processing time and memory to process the instructions and data. As the processing power and memory are the most precious resources of a computer, a program should be laid out in such a manner that it utilizes the least amount of memory and processing time.
- **Structural:** To develop a program, the task must be broken down into a number of subtasks. These subtasks are developed independently, and each subtask is able to perform the assigned job without the help of any other subtask. If a program is developed structurally, it becomes more readable, and the testing and documentation process also gets easier.
- **Flexibility:** A program should be flexible enough to handle most of the changes without having to rewrite the entire program. Most of the programs are developed for a certain period and they require modifications from time to time. For example, in case of payroll management, as the time progresses, some employees may leave the company while some others may join. Hence, the payroll application should be flexible enough to incorporate all the changes without having to reconstruct the entire application.
- **Generality:** Apart from flexibility, the program should also be general. Generality means that if a program is developed for a particular task, then it should also be used for all similar tasks of the same domain. For example, if a program is developed for a particular organization, then it should suit all the other similar organizations.
- **Documentation:** Documentation is one of the most important components of an application development. Even if a program is developed following the best programming practices, it will be rendered useless if the end user is not able to fully utilize the functionality of the application. A well-documented application is also useful for other programmers because even in the absence of the author, they can understand it.

1.

Program Development Cycle

1. Analyze – Define the problem.

You must have a clear idea of what data (or input) is given and the relationship between the input and the desired output.

2. Design – Plan the solution to the problem.

Find a logical sequence of precise steps that solve the problem (aka the algorithm). The logical plan may include flowcharts, pseudocode, and top-down charts.

3. Design the interface – Select objects (text boxes, buttons, etc.).

Determine how to obtain input and how the output will be displayed. Objects are created to receive input and display output. Appropriate menus, buttons, etc. are created to allow user to control the program.

4. Code – Translate algorithm into a programming language.

During this stage that program is written.

5. Test and debug – Locate and remove errors in program.

Testing is the process for finding errors. Debugging is the process for correcting errors.

6. Complete the documentation – Organize all materials that describes the program.

Documentation is necessary to allow another programmer or non-programmer to understand the program. Internal documentation, known as comments, are created to assist a programmer. An instruction manual is created for the non-programmer. Documentation should be done during the coding stage.